CYBERSECURITY INTERN REPORT AT SHADOWFOX

Cyber Security Internship: Level 1 (Beginner) Tasks

Target: testphp / testasp.vulnweb.com (Acunetix test site)

Name: Bithin Krishna Radhakrishnan

Batch: October 04

Task Level: Beginner

Table of Contents

List of Figures

Figure No	Name	Page No
1	Nmap service scan (nmap -sV)	
2	Gobuster directory scan (complete run)	
3	Wireshark HTTP stream showing plaintext login credentials	

1. Introduction and Information About the Report and the Machine

1.1 Introduction

Cyber security depends on knowing how a system can be probed and where it is exposed before an attacker finds those weaknesses. During this internship at ShadowFox I completed the Beginner level tasks, which run a set of basic security assessments against the deliberately vulnerable web application http://testasp.vulnweb.com/. The target is a test site provided by Acunetix for use with its vulnerability scanner, so every activity in this report was carried out against a system that permits testing.

1.2 Information About the Report

This report covers three tasks:

- Task 1: Identifying open ports and services on the target using Nmap.
- Task 2: Enumerating directories on the target using Gobuster with a common wordlist.
- Task 3: Logging in to the target while capturing traffic in Wireshark to recover the credentials sent over the network.

For each task, the report gives the attack name, severity, steps with screenshots, impact, mitigation steps, references, and the resources used.

1.3 Required Machine

- Standard computer system with network connectivity.
- Kali Linux (Linux operating system) with a browser and the tools listed in Section 4.

2. Task 1: Find All the Ports That Are Open on the Website

Target: http://testasp.vulnweb.com/ (resolves to 44.238.29.244)

Attack name: Port scanning and service enumeration

Severity: High, score 7.0 to 8.9. Reason: the open HTTP service on port 80 is unencrypted, so credentials and session data sent over it can be read by anyone on the network path. Port 25 is also open, and both services expose information an attacker can use to plan further attacks.

2.1 Steps to Reproduce

Step 1: Target identification. The host was resolved with a DNS lookup. The reverse DNS record shows the server is hosted on Amazon EC2 in the us-west-2 region.

Step 2: Service and version scan. I ran an Nmap service detection scan against the target with the command below. The -sV flag asks Nmap to probe open ports to identify the service running on each.

Command: nmap -sV testasp.vulnweb.com



Fig (1): Nmap service scan (nmap -sV) showing ports 25 and 80 open

Results: Nmap reports 998 ports filtered (no response). Two ports are open, as shown in the table below.

Port/Protocol	State	Service	Version
25/tcp	open	tcpwrapped	Not identified
80/tcp	open	tcpwrapped	Not identified

Port 80 hosts an HTTP service, which is the web application itself. The Wireshark capture in Task 3 shows the server responds as Microsoft-IIS/8.5 with X-Powered-By: ASP.NET, which identifies the server software even though Nmap could not identify the service on the port.

Note: A later plain nmap run returned all 1000 scanned ports as filtered. The -sV scan is treated as the reliable result, since it returned open ports and a server response.

2.2 Analysis

Port 80 is open and serves plain HTTP. HTTP traffic is not encrypted, so anyone able to observe the traffic can read it. Port 25 is open and is normally used for SMTP mail transfer, so it should be confirmed as needed. The filtered ports indicate that a firewall or other filtering is in front of the host, which limits the exposed attack surface.

2.3 Impact

- Unencrypted HTTP can be intercepted, leading to credential theft and session hijacking, including through man-in-the-middle attacks.

- An open port that is not needed gives attackers an additional entry point to probe for vulnerabilities.
- Exposed service details, such as server software and version, help attackers choose exploits that match the target.

2.4 Mitigation Steps

- Redirect all HTTP traffic on port 80 to HTTPS and enforce HSTS so browsers only connect over encrypted channels.
- Close any port that is not required for business use, and confirm whether port 25 needs to be publicly reachable.
- Keep the web server and its components patched, and remove or restrict banner information such as X-Powered-By headers.
- Restrict access with firewall rules so that only required sources can reach each open port.
- Run regular port scans against the perimeter to confirm that only approved services are exposed.

3. Task 2: Brute Force the Website and Find the Directories

Target: http://testasp.vulnweb.com

Attack name: Directory brute forcing (directory enumeration)

Severity: Medium, score 4.0 to 6.9. Reason: the scan found several directories that reveal the application structure, and the cgi-bin directory and robots.txt point to areas an attacker would examine more closely. The directories found did not expose credentials or confidential files in this test, which is why the rating is lower than the other tasks.

3.1 Steps to Reproduce

Step 1: Target URL. The target URL is http://testasp.vulnweb.com.

Step 2: Directory brute force. I used Gobuster in directory mode with 10 threads and the common.txt wordlist from the dirb package. Gobuster requests each word from the list and reports the paths that return a response other than 404.

Command: gobuster dir -u http://testasp.vulnweb.com -w /usr/share/dirb/wordlists/common.txt

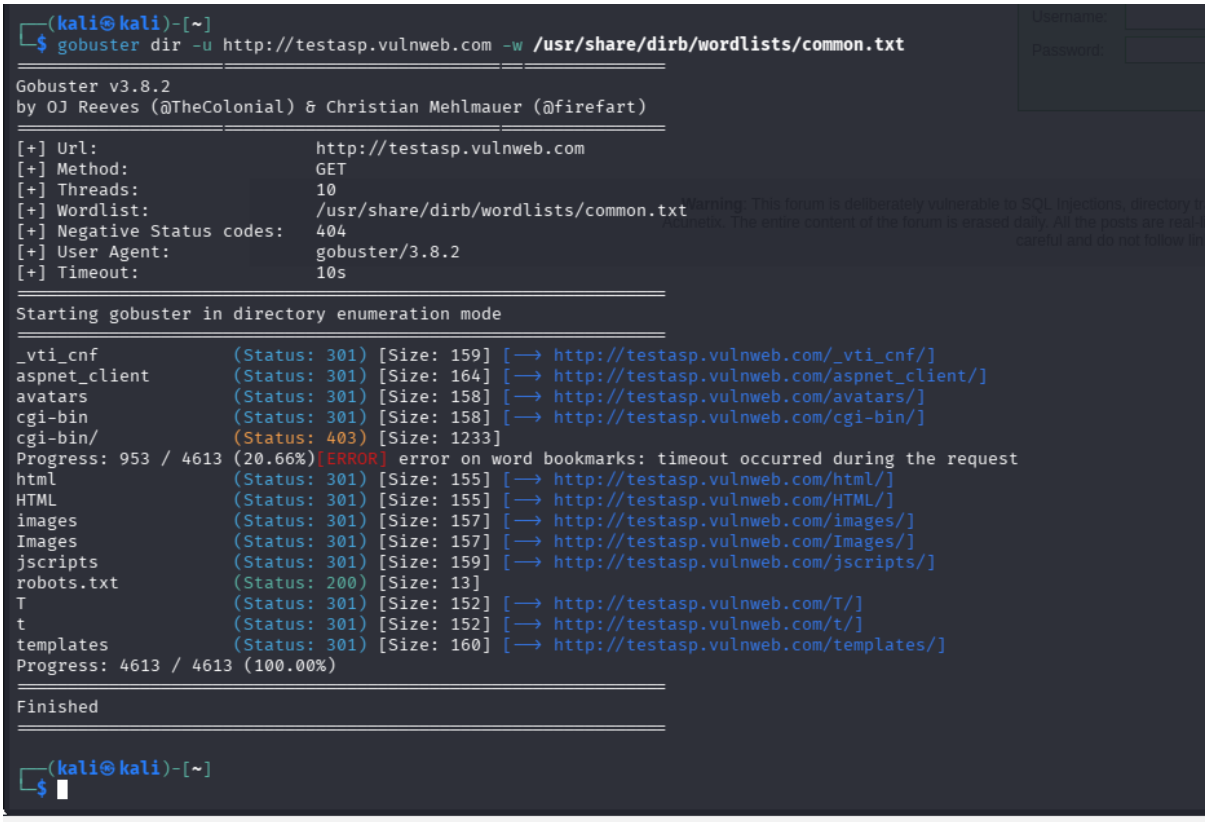


Fig (2): Gobuster directory scan, complete run (4613 / 4613 words)

Results: The following paths were found:

Path	Status	Redirects to / Notes
/_vti_cnf/	301	FrontPage metadata directory
/aspnet_client/	301	ASP.NET client-side resources
/avatars/	301	User image directory
/cgi-bin/	301 / 403	Script directory; access forbidden on /cgi-bin/
/html/, /HTML/	301	Content directories (case variants)
/images/, /Images/	301	Image directories (case variants)
/jscripts/	301	JavaScript directory
/templates/	301	Template directory
/T/, /t/	301	Short single-letter directories

/robots.txt	200	Crawler instructions file, publicly readable

Scan note: The scan logged one timeout error at word 953 of 4613 (about 21%). The run then continued and completed all 4613 words, as shown in Figure 2. The results above cover the full wordlist.

3.2 Analysis

The most useful finding is robots.txt, which is publicly accessible and can list paths the site owner wants crawlers to avoid. The 301 responses show that most directories exist, and the 403 on /cgi-bin/ shows that the directory exists but access is restricted. The duplicate case variants (html/HTML, images/Images, t/T) suggest the server is case-insensitive, which is a configuration detail worth noting.

3.3 Impact

- Exposed directory names reveal the application structure and technology, which helps an attacker plan targeted attacks.
- Publicly readable robots.txt can point attackers to paths the owner would prefer to keep hidden.
- Directories such as cgi-bin are common places for misconfigured scripts, so they need closer review even when access is currently denied.

3.4 Mitigation Steps

- Disable directory listing on the web server and return generic error pages for unused paths.
- Review what robots.txt reveals and avoid listing sensitive paths in it, since it is public.
- Restrict cgi-bin and other script directories to only what is required, and keep scripts updated.
- Remove unused directories and default files such as _vti_cnf that are not needed in production.
- Enforce rate limiting and monitor for repeated requests that indicate brute forcing.

4. Task 3: Log In to the Website, Capture Traffic in Wireshark, and Find the Credentials

Target: http://testasp.vulnweb.com/Login.asp

Attack name: Network sniffing of plaintext credentials

Severity: High, score 7.0 to 8.9. Reason: credentials were transmitted in plaintext over HTTP, so anyone on the network path can capture them and reuse them to access user accounts.

4.1 Steps to Reproduce

Step 1: Start the capture. I opened Wireshark on the eth0 interface and started a capture. I also used Burp Suite Community Edition alongside the browser to observe requests.

Step 2: Log in. I opened the login page at http://testasp.vulnweb.com/Login.asp in a browser and submitted a login attempt using the username User and a password (masked in this report).

Step 3: Filter the traffic. I applied the display filter tcp.stream eq 48 to isolate the TCP session containing the POST request. The HTTP request to /Login.asp was then followed with Follow > HTTP Stream.

Step 4: Read the credentials. The HTTP stream shows the form data sent in clear text in the request body.

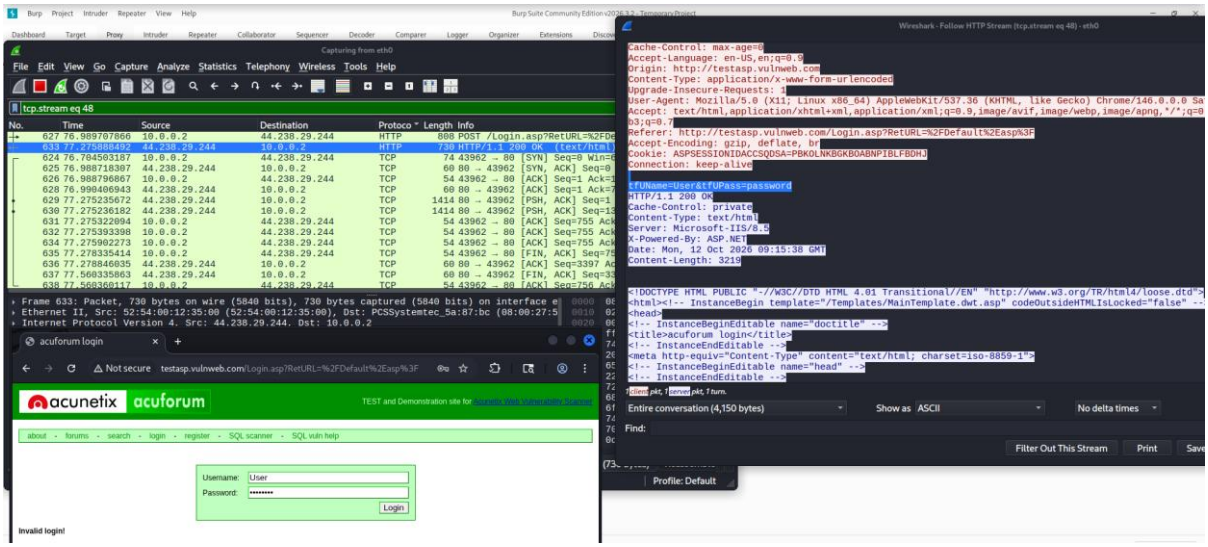


Fig (3): Wireshark HTTP stream (tcp.stream eq 48) showing plaintext credentials in the POST body

Results: The captured POST request to /Login.asp contains the following form data:

Field	Value
tfUName	User
tfUPass	p******** (masked)

The server responded with HTTP/1.1 200 OK and the page showed "Invalid login!", so the credentials were exposed but did not grant access on this attempt. The response was also sent over plain HTTP, with Server: Microsoft-IIS/8.5 and X-Powered-By: ASP.NET in the headers.

4.2 Analysis

The login form is served and submitted over HTTP without encryption. Because the request body contains the username and password in readable form, any device on the same network path, or any attacker with access to traffic, can recover them. The capture confirms that the application does not protect credentials in transit.

4.3 Impact

- Captured credentials can be reused to log in to user accounts, leading to unauthorised access and misuse of personal data.
- If the same password is used on other services, the exposure extends beyond this website.
- Accounts linked to financial or personal information could be used for fraud.
- The exposure damages user privacy and can create legal and compliance problems for the operator.

4.4 Mitigation Steps

- Serve the login page and all authentication requests over HTTPS with a valid TLS certificate, and reject plain HTTP submissions.
- Enable HSTS so browsers refuse to send credentials over an unencrypted connection.
- Add multi-factor authentication so that captured passwords alone are not enough to access an account.
- Store passwords as salted hashes on the server and enforce a strong password policy.
- Monitor the network for unusual login activity and use intrusion detection to flag repeated or anomalous authentication attempts.
- Educate users not to reuse passwords across services.

5. Resources Used

- Kali Linux operating system.
- Nmap 7.99 for port scanning and service detection.
- Gobuster v3.8.2 for directory enumeration, with the dirb common.txt wordlist.
- Wireshark for packet capture and HTTP stream analysis.
- Burp Suite Community Edition for observing requests.
- Chromium browser for accessing the target login page.
- Target: testasp.vulnweb.com, the Acunetix test application.

6. References

- Nmap Project, Nmap Reference Guide: https://nmap.org/book/man.html
- OJ Reeves and Christian Mehlmauer, Gobuster project: https://github.com/OJ/gobuster
- Wireshark Foundation, Wireshark User Guide: https://www.wireshark.org/docs/
- MITRE, CWE-319: Cleartext Transmission of Sensitive Information: https://cwe.mitre.org/data/definitions/319.html
- OWASP, Transport Layer Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html
- OWASP, Testing for Weak Transport Layer Security (WSTG): https://owasp.org/www-project-web-security-testing-guide/
- Acunetix, Test site information: http://testasp.vulnweb.com/
- Mozilla Developer Network, Strict-Transport-Security (HSTS): https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security